Digital RF



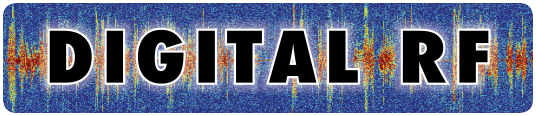
Ryan Volz

HamSCI Meteor Scatter Working Group

Digital RF

Format and software for RF samples

- Geared toward continuous recording with known timing
- Common workflow:
  - Record from time-synchronized SDR
  - Analyze data in quasi-realtime
  - Copy useful data to archive
  - Discard old data when space is needed
- Source/sink blocks for GNU Radio
- Included tools:
  - Ringbuffer, mirror, copy/move by time subset
  - Command line plotting



https://github.com/MITHaystack/digital_rf

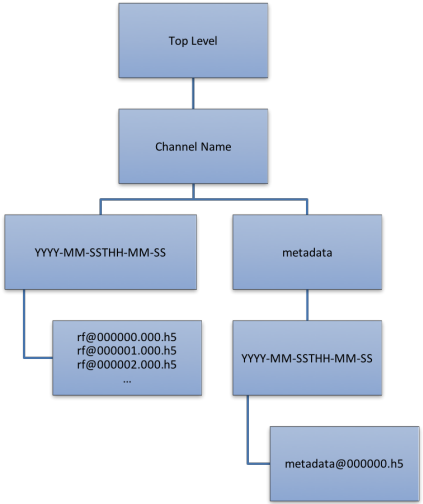
```
$ conda install -c conda-forge digital_rf
```

or

```
$ pip install digital_rf
```

Digital RF: The Format

- Related data goes under a top-level “experiment” directory
- Channels of data have a given (short) name
  - e.g. “ch0” or “vert_pol”
- Fixed sample rate for each channel
- Samples are referenced and stored by their global sample index
  - Index = Seconds since epoch * Sample rate
- Storage broken into separate HDF5 files with timestamped names in timestamped directories
  - Configurable file cadence and subdirectory cadence



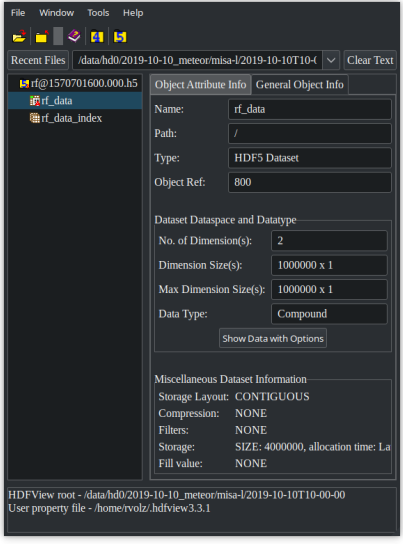
Digital RF: HDF5

HDF5 format at file level

- Builds in basic metadata descriptors like sample type and array shape
  - Bare binary would require external description
- Optional compression and chunked storage

HDF5 files have two datasets

- `rf_data` (num_samples) x (num_subchannels)
  - Binary data chunk
  - Subchannels are interleaved
  - IQ samples are interleaved
- `rf_data_index` (num_blocks) x (2)
  - Maps global index to index within `rf_data`
  - e.g. [(15671057000000, 0),
    (15671057002000, 1000)]



Digital RF: Usage in Python

Basic Python usage

```python
from datetime import datetime
import digital_rf as drf

# Create reader object
example = drf.DigitalRFReader("data/example")

# List channels
print(example.get_channels())

# Get data bounds and properties for channel 'ch0'
start_idx, end_idx = example.get_bounds("ch0")
properties = example.get_properties("ch0")
sps = properties["samples_per_second"]

# Read first 10 samples from channel 'ch0'
data = example.read_vector(start_idx, 10, "ch0")

# Read samples from a specific time
dt = datetime.fromisoformat("2019-08-29T19:08:20")
sample_idx = drf.util.time_to_sample(dt, sps)
data = example.read_vector(sample_idx, 1000, "ch0")
```

